Chapitre 5:
La récursivité

I. Définition

La récursivité est un outil très puissant en programmation. Lorsqu'elle est bien utilisée, elle rend la programmation plus facile. C'est avant tout une méthode de résolution de problème.

On distingue plusieurs types de récursivité :

- **récursivité directe** : lorsqu'un module fait appel à lui-même.
- **récursivité indirecte ou croisée** : lorsqu'un module A fait appel à un module B qui appelle A.
- **récursivité circulaire** : lorsqu'un module A fait appel à un module B, B fait appel à un module C qui appelle A.

I. Définition

Illustration

Cas 1 : récursivité directe

```
Procédure ProcRecursive
(paramètres)
Début
      …
      ProcRecursive (valeurs)
      …
Fin proc
```

Cas 2 : récursivité indirecte

```
Procédure A (paramètres)
Début
          …
          B (valeurs)
{appel de la procédure B dans A}
          …
Fin proc

Procédure B (paramètres)
Début
             …
            A (valeurs)
{appel de la procédure A dans B}
             …
Fin proc
```

I. Définition et Classifications

Concernant les méthodes, on peut trouver d'autres classifications de récursivité :

récursivité non terminale : Une méthode récursive est dite non terminale si le résultat de l'appel récursif est utilisé pour réaliser un traitement (en plus du retour du module).

récursivité terminale : Une méthode récursive est dite terminale si aucun traitement n'est effectué à la remontée d'un appel récursif (sauf le retour du module).

I. Définition et Classifications

NB. Un algorithme est dit récursif terminal s'il ne contient aucun traitement après un appel récursif.

```
Procédure ALGO (X) // Liste des paramètres
  Si (condition) alors
      <Traitement 1> // Traitement de terminaison (dépendant de X)
  Sinon   //cas général:  condition portant sur X
   <Traitement 2> // traitement de base de l'algorithme (dépendant de X)
   ALGO (F(X)) // F(X) représente la transformation des paramètres
  // Rien  !!!!!  pas de code après l'appel récursif
  Finsi
Fin Proc.
```

II. Étude d'un exemple : la fonction factorielle

Dénotée par n ! (se lit factorielle n), c'est le produit de nombres entiers positifs de 1 à n inclus.

Exemples :

4 ! = 1*2*3*4,

5 ! = 1*2*3*4*5,

Noter que 4 ! peut s'écrire 4*3*2*1 = 4 * 3 ! et que 5 ! peut s'écrire 5*4*3*2*1 = 5 * 4 !

➔ **On peut conclure que : n! = 1 si (n=1 ou n=0)**

n! = n* (n-1) ! si non

II. Étude d'un exemple : la fonction factorielle

A. Solution non terminale

```
int facto(int n)
{
      if (n == 1 || n == 0)
            return 1;
      else
            return n * facto(n-1);
}
```

Illustration

Chaque cas est réduit à un cas plus simple. Le calcul de 4! se ramène à celui de 3!, le calcul de 3! se ramène au calcul de 2! … jusqu'à arriver à 1! qui donne directement 1.

Après on fait un retour arrière. Le résultat d'une ligne i sert au calcul de la ligne i-1

II. Étude d'un exemple : la fonction factorielle

A. Solution non terminale

➔**Illustration du mécanisme de fonctionnement:**
Considérons le calcul de 4! par la fonction récursive définie ci-dessus :

facto(4) renvoie 4 * facto(3)
facto(3) renvoie 3 * facto(2)
facto(2) renvoie 2 * facto(1)
facto(1) renvoie 1 (arrêt de la récursivité)
facto(2) renvoie 2 * 1 = 2
facto(3) renvoie 3 * 2 = 6
facto(4) renvoie 4 * 6 = 24

II. Étude d'un exemple : la fonction factorielle

B. Solution terminale

```
int facto(int n, int resultat) {
      if (n == 1 || n == 0 )
            return resultat;
      else
            return facto(n-1, n*resultat) ;
}
```

Exemple: 4!:

facto(4, 1) renvoie facto(3, 4)
facto(3, 4) renvoie facto(2, 12)
facto(2, 12) renvoie facto(1, 24)
facto(1, 24) renvoie 24

III. Conseils d'écriture d'une fonction récursive

Ces conseils sont illustrés par l'exemple suivant :
Écrire une fonction récursive permettant de calculer la somme des chiffres d'un entier n positif
Exemple : n = 528, donc la somme des chiffres de n est 15

1. Observer le problème afin de :

a) Paramétrer le problème : on détermine les éléments dont dépend la solution et qui caractérisent la taille du problème.

b) Décrire la condition d'arrêt : quand peut-on trouver "facilement" la solution ? **(une solution triviale)** :
Si on a le choix entre n = 528 et n = 8, il est certain qu'on choisit n = 8. La somme des chiffres de 8 est 8.

➔**Conclusion :** Si n a un seul chiffre, on arrête. La somme des chiffres est n lui-même.

III. Conseils d'écriture d'une fonction récursive

c) réduire le problème à un problème d'ordre inférieur pour que la condition d'arrêt soit atteinte un moment donné :

somChif (528) =8+somChif(52)

= 8 + (2 + somChif(5)) = 8 + (2 + 5)

2. Écriture de la fonction :

```
int somChif(int n) {
    if (n < 10)  // condition d'arrêt : un seul chiffre
        return n;
    else              // réduction du problème
        return (n % 10) + somChif(n / 10);
}
```

III. Conseils d'écriture d'une fonction récursive

Exercice :

Illustrer les conseils précédents pour écrire une fonction récursive qui permet de calculer le produit de deux entiers positifs a et b sans utiliser l'opérateur de multiplication (*).

Solution :

a) la solution de ce problème dépond des deux opérandes n1 et n2

b) Si vous avez le choix entre :12 x 456, 12 x 0 , 12 x 1 Lesquels des trois calculs sont le plus simple ?

c) 12 x 9 = 12 + 12 + 12 + ... + 12 (9 fois)
= 12 + (12 + 12 + ... + 12) (8 fois)
= 12 + 12 x 8

etc ...

III. Conseils d'écriture d'une fonction récursive

Version récursive non terminale

```
#include <stdio.h>

int produit(int a, int b) {
    if (b == 0)    // condition d'arrêt
        return 0;
    else           // réduction du problème
        return a + produit(a, b - 1);
}

void main() {
    int a = 12, b = 9;
    int res = produit(a, b);
    printf("Le produit de %d et %d est : %d\n", a, b, res);
}
```

III. Conseils d'écriture d'une fonction récursive

Version récursive terminale

```
#include <stdio.h>

int produit_terminale(int a, int b, int acc){
    if(b == 0)
        return acc;
    else
        return produit_terminale(a, b-1, acc+a);
}

void main() {
    int a = 12, b = 9;
    int res = produit_terminale(a, b, 0);

    printf("Le produit de %d et %d est : %d\n", a, b, res);
}
```